中山大學
SUN YAT-SEN UNIVERSITY

Lecture 5
Linked Stacks and Linked Queues



Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University
zhangzizhen@gmail.com

Outline

- Pointers and Linked Structures
- Linked Stacks
- Linked Stacks with Safeguards
- Linked Queues

Pointers

- Pointer: A pointer is a programming language object, whose value refers directly to (or "points to") another value stored elsewhere in the computer memory using its address.
- Pointers to data significantly improve performance for repetitive operations such as traversing strings, lookup tables and tree structures. In particular, it is often much cheaper in time and space to copy and dereference pointers than it is to copy and access the data to which the pointers point.

Pointers

- One major problem with pointers is that as long as they can be directly manipulated as a number, they can be made to point to unused addresses or to data which is being used for other purposes.
- Many languages, including most functional programming languages and recent imperative languages like Java, replace pointers with a more opaque type of reference, typically referred to as simply a reference, which can only be used to refer to objects and not manipulated as numbers, preventing this type of error. Array indexing is handled as a special case.

Key Terms

- **Contiguous:**

  Next to each other, touching, adjoining; used in contrast to *linked.*

- **Linked list:**

  A list in which each entry contains a pointer giving the location of the next entry.

- **Automatic object:**

  An object that exists as long as the block of program declaring it is active; referenced by giving it a name when writing the program.

- **Dynamic object:**

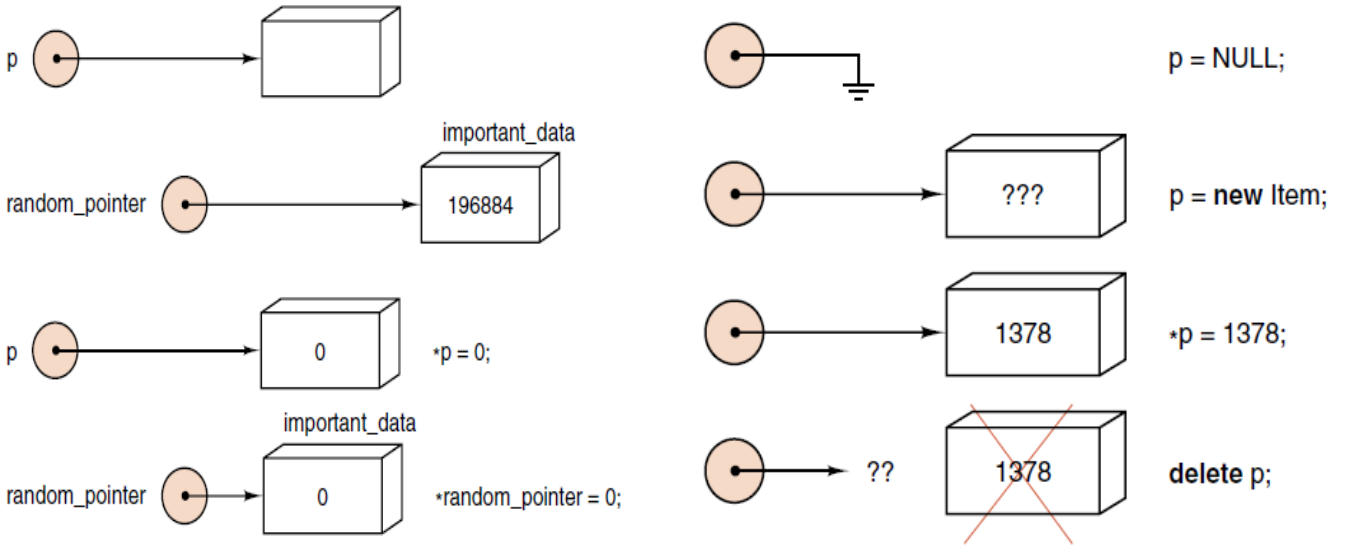
  An object that is created (and perhaps destroyed) while the program is running; accessed indirectly via pointers.

Pointers in C++

- **Creating dynamic objects:**
  - item_ptr = **new** Item; creates a new dynamic object of type Item and assigns its location to the pointer variable item_ptr.
  - The dynamic objects that we create are kept in an area of computer memory called the *free store (or the heap).*
- **Deleting dynamic objects:**
  - **delete** item_ptr; disposes of the dynamic object to which item_ptr points and returns the space it occupies to the free store so it can be used again.
  - After this **delete** statement is executed, the pointer variable item_ptr is undefined and so should not be used until it is assigned a new value.
- **Following pointers:**
  - *item_ptr denotes the object to which item_ptr points. The action of taking *item_ptr is called "**dereferencing** the pointer *item_ptr."

Pointers in C++

- **NULL pointers:**
  - If a pointer variable item_ptr has no dynamic object to which it currently refers, then it should be given the special value NULL.
- **Undefined pointers versus NULL pointers:**
  - If the value of item_ptr is undefined, then item_ptr might point to any random location in memo



The Basics of Linked Structures

- A linked structure is made up of nodes, each containing both the information that is to be stored as an entry of the structure and a pointer telling where to find the next node in the structure. We shall refer to these nodes making up a linked structure as the ***nodes*** of the structure, and the pointers we often call ***links***. Since the link in each node tells where to find the next node of the structure, we shall use the name next to designate this link.

```
struct Node {
   // data members
   Node_entry entry;
   Node *next;
   // constructors
   Node( );
   Node(Node_entry item, Node *add_on = NULL);
};
```

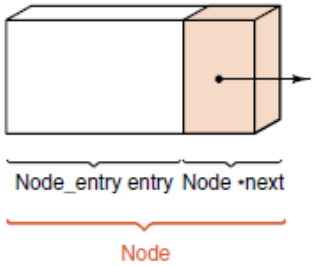
Node Constructors

- First form:

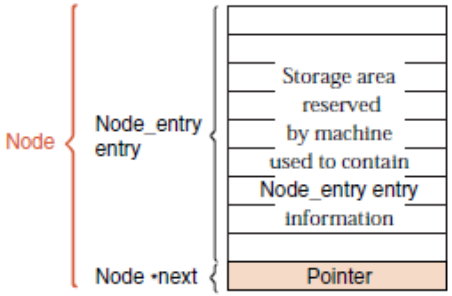
```
Node :: Node( )
{
   next = NULL;
}
```

- The second form accepts two parameters for initializing the data members.

```
Node :: Node(Node_entry item, Node *add_on)
{
   entry = item;
   next = add_on;
}
```



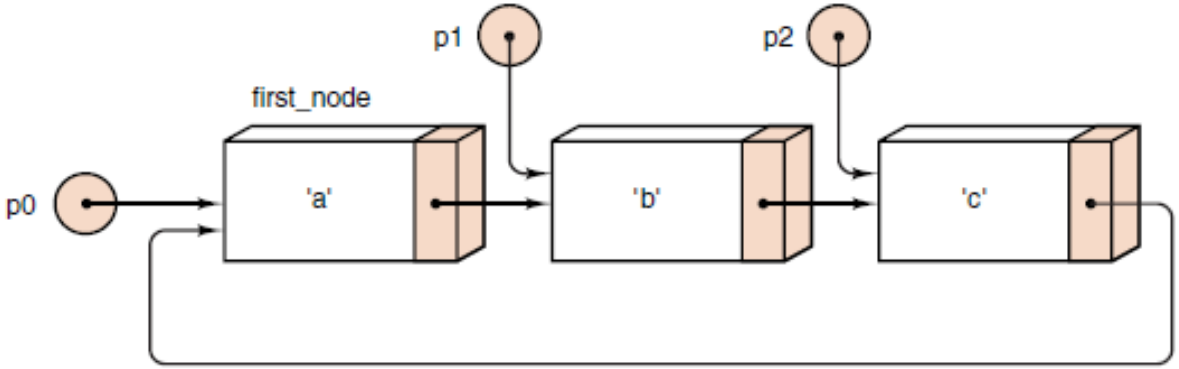
(a) Structure of a Node



(b) Machine storage representation of a Node

Node Constructors

```
Node first_node('a');       // Node first node stores data 'a'.
Node *p0 = &first_node;     // p0 points to first Node.
Node *p1 = new Node('b');   // A second node storing 'b' is created.
p0->next = p1;              // The second Node is linked after first node.
Node *p2 = new Node('c', p0);   // A third Node storing 'c' is created.
                // The third Node links back to the first node, *p0.
p1->next = p2;              // The third Node is linked after the second Node.
```



Typedef

- typedef is a keyword in the C and C++ programming languages. The purpose of typedef is to form complex types from more-basic machine types and assign simpler names to such combinations.
- Indicating what a variable represents

  typedef int Stack_entry; or, typedef char Stack_entry;
- Using typedef with pointers

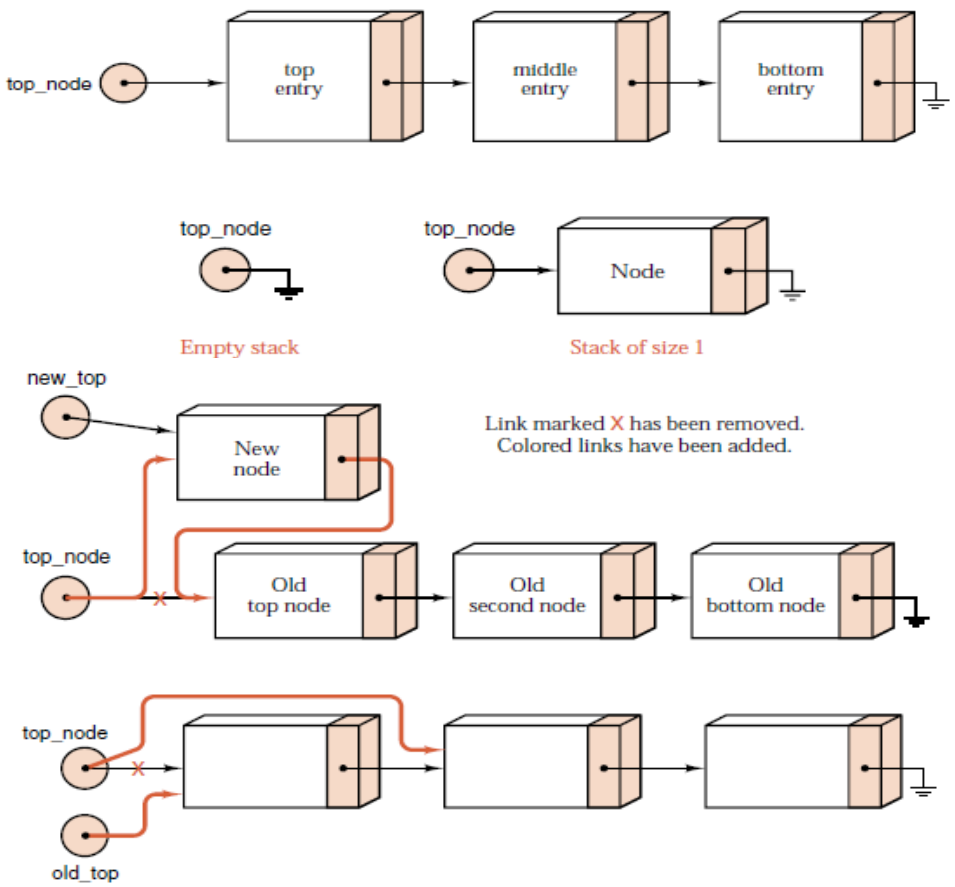
```
typedef int *intptr;    // type name: intptr
                        // new type: int*

intptr ptr;             // same as: int *ptr
```

- Using typedef with structure pointers

```
typedef struct Node* NodePtr;
...
NodePtr startptr, endptr, curptr, prevptr, errptr, refptr;
```

Linked Stacks



Class Declaration for Linked Stack

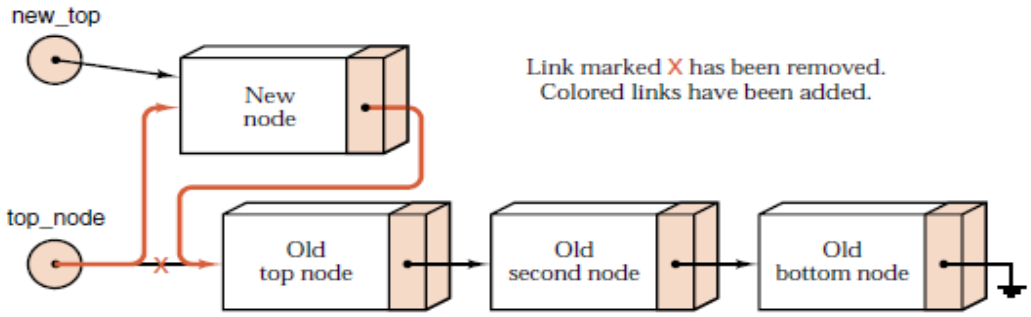
```
class Stack {
public:
  Stack( );
  bool empty( ) const;
  Error_code push(const Stack_entry &item);
  Error_code pop( );
  Error_code top(Stack_entry &item) const;
protected:
  Node *top_node;
};
```

- 用类定义链栈的好处：封装性、从逻辑上可区分栈和栈顶两个概念

Linked Stack Operations

- **Pushing a Linked Stack**

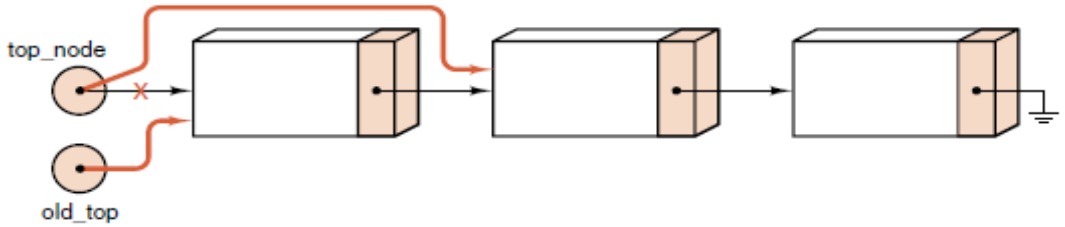
```
Error_code Stack :: push(const Stack_entry &item)
/* Post: Stack entry item is added to the top of the Stack; returns success
or returns a code of overflow if dynamic memory is exhausted. */
{
   Node *new_top = new Node(item, top_node);
   if (new_top == NULL) return overflow;
   top_node = new_top;
   return success;
}
```



Linked Stack Operations

- **Popping a Linked Stack**

```
Error_code Stack :: pop( )
/* Post: The top of the Stack is removed. If the Stack is empty the
method returns underflow; otherwise it returns success. */
{
   Node *old_top = top_node;
   if (top_node == NULL) return underflow;
   top_node = old_top->next;
   delete old_top;
   return success;
}
```



Linked Stacks with Safeguards

- Example:

```
for (int i = 0; i < 1000000; i++) {
   Stack small;
   small.push(some_data);
}
```

- Client code can apply the methods of linked stacks in ways that lead to the accumulation of garbage or that break the encapsulation of Stack objects.
- C++ provides three devices (additional class methods) to alleviate these problems:
  - destructors,
  - copy constructors
  - overloaded assignment operators

The Destructor

- A destructor is automatically executed on objects of the class when the objects go out of scope.
- It is often used to clean up the space that would otherwise become garbage.

```
Stack :: ~Stack( )    // Destructor
/* Post: The Stack is cleared. */
{
   while (!empty( ))
      pop( );
}
```

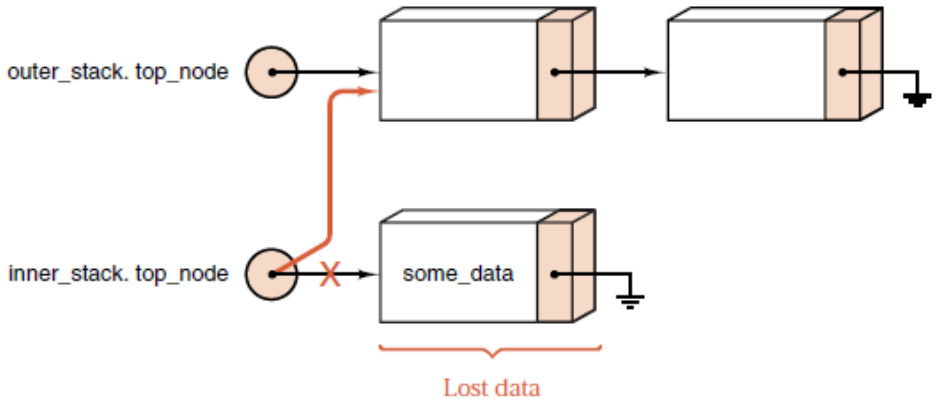
Policy

Every linked structure should be equipped with a destructor to clear its objects before they go out of scope.

Dangers in Assignment

- Example

```
Stack outer_stack;
for (int i = 0; i < 1000000; i++) {
   Stack inner_stack;
   inner_stack.push(some_data);
   inner_stack = outer_stack;
}
```



Overloading the Assignment Operator

- Misbehaviors
  - Lost data space.
  - Two stacks have shared nodes.
  - The destructor on inner stack deletes outer stack.
  - Such a deletion leaves the pointer outer stack.top node addressing what a random memory location.
- Prototype

  void Stack :: **operator** = (const Stack &original);

  - This declares a Stack method called operator = , the overloaded assignment operator, that can be invoked with **x.operator** = (y); or x = y;
- Implementation outline
  - Make a copy of the data stacked in the calling parameter.
  - Clear out any data already in the Stack object being assigned to.

Overloaded Assignment of Linked Stacks

```
void Stack :: operator = (const Stack &original) // Overload assignment
/* Post: The Stack is reset as a copy of Stack original. */
{
   Node *new_top, *new_copy, *original node = original.top_node;
   if (original_node == NULL) new_top = NULL;
   else { // Duplicate the linked nodes
      new_copy = new_top = new Node(original_node->entry);
      while (original_node->next != NULL) {
         original_node = original_node->next;
         new_copy->next = new Node(original_node->entry);
         new_copy = new_copy->next;
      }
   }
   while (!empty( )) // Clean out old Stack entries
      pop( );
   top_node = new_top; // and replace them with new entries.
}
```

The Copy Constructor

- Example

```
void destroy_the_stack (Stack copy) {
}
int main( ) {
   Stack vital_data;
   destroy_the_stack(vital_data);
}
```

The Stack copy shares its nodes with the Stack vital_data, and therefore when a Stack destructor is applied to copy, at the end of the function, vital_data is also destroyed.

- Prototype

```
Stack :: Stack(const Stack &original);
```

- Implementation outline
  - Deal with the case of copying an empty Stack.
  - Copy the first node.
  - Run a loop to copy all of the other nodes.

Linked Stack Copy Constructor

```
Stack :: Stack(const Stack &original) // copy constructor
/* Post: The Stack is initialized as a copy of Stack original. */
{
  Node *new_copy, *original_node = original.top_node;
  if (original_node == NULL) top_node = NULL;
  else { // Duplicate the linked nodes.
    top_node = new_copy = new Node(original_node->entry);
    while (original_node->next != NULL) {
      original_node = original_node->next;
      new_copy->next = new Node(original_node->entry);
      new_copy = new_copy->next;
    }
  }
}
```

Policy

For every linked class, include a copy constructor, or
warn clients that objects are copied with reference semantics.

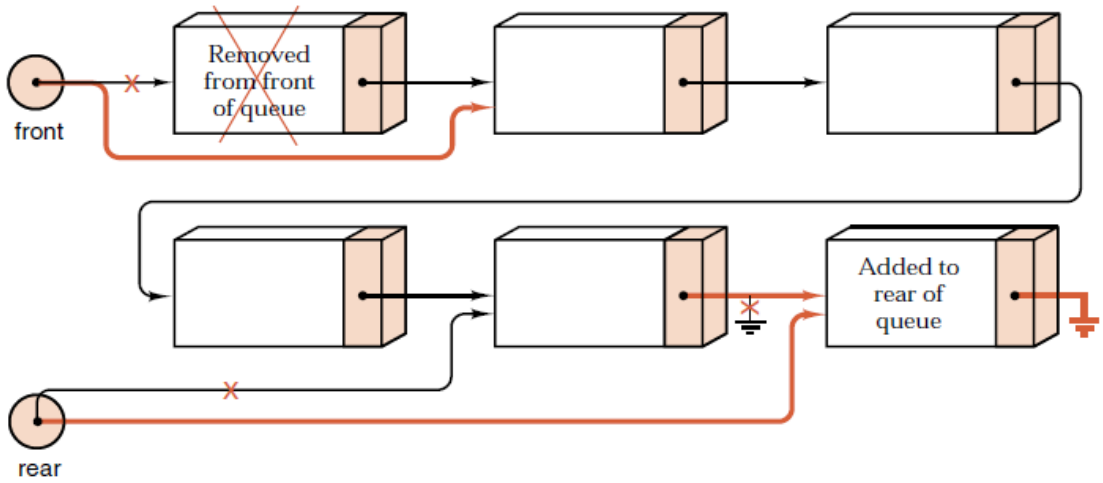
Modified Linked-Stack Specification

```
class Stack {
public:
   // Standard Stack methods
   Stack( );
   bool empty( ) const;
   Error_code push(const Stack_entry &item);
   Error_code pop( );
   Error_code top(Stack_entry &item) const;
   // Safety features for linked structures
   ~Stack( );
   Stack(const Stack &original);
   void operator = (const Stack &original);
protected:
   Node *top_node;
};
```

Linked Queues

```
class Queue {
public:
   // standard Queue methods
   Queue( );
   bool empty( ) const;
   Error_code append(const Queue_entry &item);
   Error_code serve( );
   Error_code retrieve(Queue entry &item) const;
   // safety features for linked structures
   ~Queue( );
   Queue(const Queue &original);
   void operator = (const Queue &original);
protected:
   Node *front, *rear;
};
```

Linked Queue Methods



Constructor:

```
Queue :: Queue( )
/* Post: The Queue is initialized to be empty. */
{
   front = rear = NULL;
}
```

Linked Queue Methods

Append an entry:

```
Error_code Queue :: append(const Queue_entry &item)
/* Post: Add item to the rear of the Queue and return a code of
success or return a code of overflow if dynamic memory is
exhausted. */
{
   Node *new_rear = new Node(item);
   if (new_rear == NULL) return overflow;
   if (rear == NULL) front = rear = new_rear;
   else {
      rear->next = new_rear;
      rear = new_rear;
   }
   return success;
}
```

Linked Queue Methods

Serve an entry:

```
Error_code Queue :: serve( )
/* Post: The front of the Queue is removed. If the Queue is empty,
return an Error code of underflow. */
{
   if (front == NULL) return underflow;
   Node *old_front = front;
   front = old_front->next;
   if (front == NULL) rear = NULL;
   delete old_front;
   return success;
}
```

Extended Linked Queues

Class definition:

```
class Extended queue: public Queue {
public:
   bool full( ) const;
   int size( ) const;
   void clear( );
   Error_code serve_and_retrieve(Queue_entry &item);
};
```

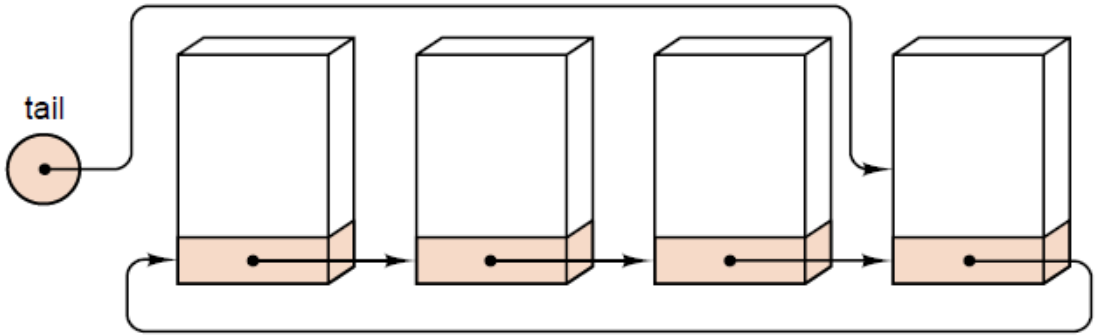
- There is no need to supply explicit methods for the copy constructor, the overloaded assignment operator, or the destructor, since the compiler calls the corresponding method of the base **Queue** object.

Extended Linked Queues

Size:

```
int Extended_queue :: size( ) const
/* Post: Return the number of entries in the Extended_queue. */
{
      Node *window = front;
      int count = 0;
      while (window != NULL) {
         window = window->next;
         count++;
      }
      return count;
}
```

Circularly Linked List



- A ***circularly linked list***, is a linked list in which the node at the tail of the list, instead of having a NULL pointer, points back to the node at the head of the list. We then need only one pointer tail to access both ends of the list, since we know that tail->next points back to the head of the list.

Thank you!

